

CODED - Cloud-Optimized Distributed Earth Data

IPFS is a viable way to preserve scientific data. A **Zarr** or **Icechunk** dataset converts to content-addressed **IPFS** in one command, pins on a service like Filebase, and re-opens straight into **xarray** - takedown-resistant, verifiable, and performant enough for real work.

Cory Levinson • Rich Signell • Brian Terry

★ An ESIP Funding Friday Project

Why preserve on IPFS?

Important environmental datasets can vanish when an institution reorganizes, loses funding, or is told to take them down. **IPFS** stores data by **content hash**, not by owner or location - so a dataset stays reachable and verifiable no matter who hosts it.

The good news: the cloud-optimized formats the community already uses - **Zarr** and **Icechunk** - drop onto IPFS **as-is**, and open right back up in **xarray**.

Step 1 • Preserve *your* dataset - CID on Filebase

Have an existing **Zarr** or **Icechunk** store you can't afford to lose? Add it to IPFS for a content ID (CID), then pin that CID on **Filebase** (IPFS + Filecoin, free to 5 GB / 500 pins) - or **any public IPFS node** - so it stays available, verifiable, and independent of any single institution.

```
# 1. add your existing store to IPFS → root CID
cid=$(ipfs add -rQ --cid-version=1 my.icechunk/)
# 2. export the whole DAG (all chunks + refs) to a CAR file
ipfs dag export $cid > my.car
# 3. pin that CAR on Filebase – import=car preserves the CID
aws s3 cp my.car s3://bucket/my.car \
  --endpoint-url https://s3.filebase.com \
  --metadata import=car # same CID, now persistent
```

The **CID is a hash of the data** — tamper-evident and location-independent. Our live ERA5 store:

bafybeiecltl3mtags2i3jeumxe5c7zuvj2r76zxwxg6tfljiouvywqzbaq

One CID = the whole queryable store. Re-chunked to 100 blocks (5 timesteps each) to fit the free-tier pin budget - **data byte-identical** (maxdiff 0.0).

Step 2 • Icechunk reads IPFS - no adapter

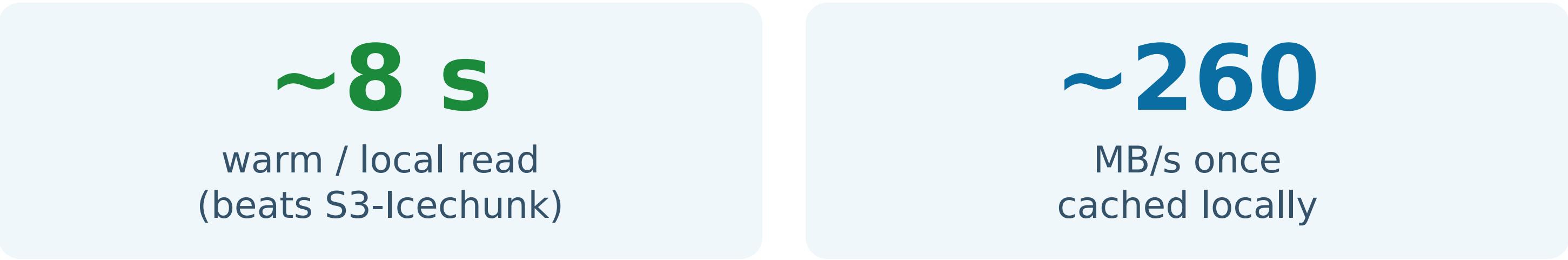
Icechunk 2.0.5 ships `http_storage(base_url=...)`, a read-only store over HTTP. Point it at an IPFS gateway path and the entire repo opens in **xarray** - the on-disk layout survives `ipfs add -r` untouched. **Zero adapter code.**

```
# whole Icechunk repo, straight from a CID
st = icechunk.http_storage("https://gw/ipfs/<CID>")
repo = icechunk.Repository.open(st)
sess = repo.readonly_session("main")
ds = xr.open_zarr(sess.store)
```

Old CID → old snapshot (reproducibility). Edit one of four chunks → **6 shared blocks reused** — cross-version dedup for free.

... and it's performant enough

We read the **same 2.08 GB ERA5** temperature slice (100 Icechunk blocks) over IPFS via `http_storage`, against a **native Icechunk-on-S3** baseline. Every path returns a bit-identical mean = **286.5062 K**.



Read path	Same-region	Cross-region
IPFS local pin / warm	~8 s	~8 s
Icechunk-on-S3 (baseline)	~14 s	~14 s
IPFS cold, first pull	~75 s	~101 s

Only the first cross-network pull of the full 2 GB is slow; it caches away and repeat reads beat S3. Pin near your readers (or co-locate a gateway) and IPFS delivers S3-class throughput.

One CID = the whole queryable store. Re-chunked to 100 blocks (~21 MB) to fit the free-tier pin budget - data byte-identical (maxdiff 0.0).

What we verified

- It survives your node.** With our EC2 node off, the CID still resolved & opened via public gateways (w3s.link, ipfs.io, dweb.link) - Filebase kept serving it.
- The data is exactly the data.** The CID is a hash of the bytes - tamper-evident, byte-identical across every read (maxdiff 0.0).
- No new tooling.** `ipfs add` to publish, `icechunk.http_storage` to read - the formats you already use, unchanged.

Bottom Line
IPFS is a practical preservation layer for cloud-native Earth data - today. Convert in one command, pin on a free-tier service, re-open in **xarray**. Your data becomes content-addressed, takedown-resistant, and reproducible - and it reads back fast enough for real analysis.

Live demo & notebooks: github.com/ESIPFed/coded-blog →

